

Agentic AI
Cheat Sheet

From zero to your first AI Agent,
step by step with real examples

Swipe to become an Agent builder →



AI Coach John FOLLOW | PROITBRIDGE Strive For Better Future

Repost

CONCEPT

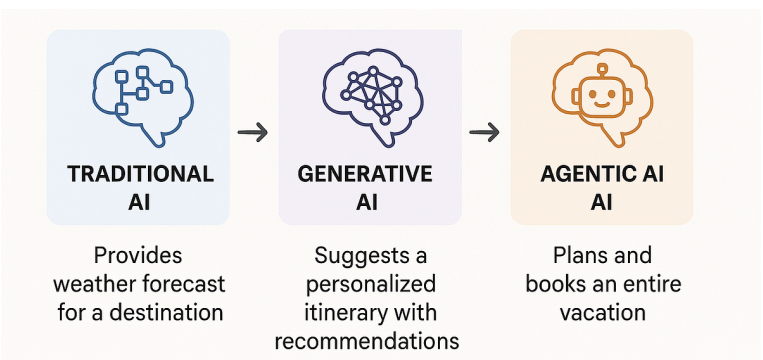
What is Agentic AI?

A regular LLM **answers questions.**
An Agentic AI **takes actions** and keeps going until the job is done.

Think of it like this: a chatbot is a librarian who looks up answers. An agent is a personal assistant who books your flight, sends the email, and checks your calendar, all on their own.

LLM VS AGENT

- ✕ LLM: one question, one answer, done
- ✓ Agent: one goal, many steps, final result
- ✓ Agent can use tools (search, code, APIs)
- ✓ Agent remembers what it did before



AI Coach John FOLLOW PROITBRIDGE Strive For Better Future Repost

CORE CONCEPTS

4 Things Every Agent Has

Every agentic system is built from these four building blocks. Learn these and everything else clicks.

Brain (LLM)

The AI model that thinks and decides what to do

Tools

Functions the agent can call: search, code, APIs

Memory

What the agent remembers from earlier steps

Goal

The task the agent is working towards

No agent works without all four. Missing one = broken agent.



AI Coach John FOLLOW

PROITBRIDGE
Strive For Better Future

Repost

HOW IT WORKS

The Think, Act, Observe Loop

Agents do not run once. They loop until the goal is done. Each loop has three steps:



EXAMPLE: "BOOK ME A FLIGHT TO NYC"

- **Think:** I need to search for flights first
- **Act:** Call search_flights("NYC", "2026-07-01")
- **Observe:** Got 5 results, cheapest is $299
- **Think:** Should I book it? Loops again...

This loop is called the ReAct pattern: Reasoning + Acting. It is the heart of every agent.


AI Coach John FOLLOW PROITBRIDGE Strive For Better Future Repost

HANDS-ON

Your First Agent in Python

Install LangChain and build a working agent in under 15 lines. It can search the web in real time.

```
pip install langchain langchain-openai duckduckgo-search

from langchain.agents import create_react_agent, AgentExecutor
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_openai import ChatOpenAI
from langchain import hub

llm    = ChatOpenAI(model="gpt-4o-mini", temperature=0)
tools  = [DuckDuckGoSearchRun()]
prompt = hub.pull("hwchase17/react")

agent  = create_react_agent(llm, tools, prompt)
runner = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = runner.invoke({"input": "What is the price of gold today?"})
print(result["output"])
```

>_ Set `verbose=True` to watch the agent think step-by-step. Great for learning!



AI Coach John FOLLOW

PROITBRIDGE Strive For Better Future

Repost

TOOLS

Giving Your Agent Superpowers

A tool is just a **Python function** with a description. The agent reads the description to decide when to call it. That is it.

```
from langchain.tools import tool

@tool
def add_numbers(a: int, b: int) -> int:
    """Adds two numbers. Use when the user asks for a sum."""
    return a + b

@tool
def get_weather(city: str) -> str:
    """Returns the current weather for a given city."""
    return f"It is sunny and 25C in {city}."

# Give both tools to your agent
tools = [add_numbers, get_weather]
```

The **description** is how the agent knows WHEN to use the tool. Write it like you are explaining it to a new colleague.


AI Coach John FOLLOW | PROITBRIDGE Strive For Better Future | Repost

MEMORY

Agent Memory Explained Simply

By default agents forget everything after each run. Add memory so they remember past steps and conversations.

3 TYPES OF MEMORY

- ✓ **Short-term** - the current conversation, stored in the prompt.
- ✓ **Long-term** - saved to a database, recalled when needed.
- ✓ **Entity** - facts about people or things ("John likes Python").

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

# Pass memory to your agent executor
runner = AgentExecutor(
    agent=agent, tools=tools,
    memory=memory, verbose=True
)
# The agent now remembers earlier messages!
```




AI Coach John FOLLOW

PROITBRIDGE Strive For Better Future

Repost

PLANNING

How Agents Plan Complex Tasks

For hard multi-step jobs, agents break them down into a plan first, then execute step by step. This is called **Plan-and-Execute**.

> **Like building a house: you do not just start laying bricks. You draw plans, buy materials, dig foundations, in order.**

```
from langchain_experimental.plan_and_execute import (
    PlanAndExecute, load_agent_executor,
    load_chat_planner
)

# Planner breaks the goal into steps
planner  = load_chat_planner(llm)

# Executor runs each step
executor = load_agent_executor(llm, tools, verbose=True)

agent = PlanAndExecute(planner=planner, executor=executor)

agent.invoke({"input":
    "Research top 3 AI trends in 2025 and write a summary"
})
```



AI Coach John FOLLOW
PROITBRIDGE Strive For Better Future
Repost

MULTI-AGENT

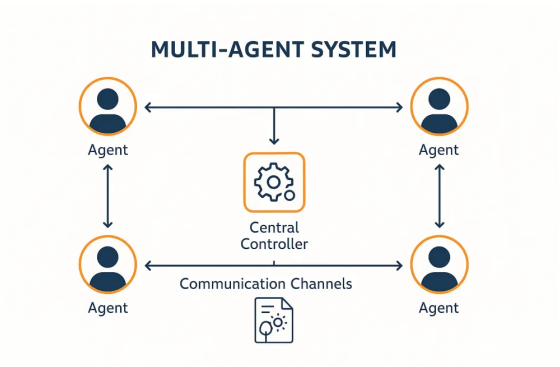
Multiple Agents Working Together

One agent for everything gets overloaded. Instead, give each agent **one job** and let them collaborate. This is a **multi-agent system**.

Like a company: a **Manager** agent delegates to a **Researcher** agent and a **Writer** agent, then reviews the result.

EXAMPLE WITH CREWAI

- **Agent 1** - Researcher: searches the web
- **Agent 2** - Writer: turns research into a blog post
- **Agent 3** - Editor: proofreads the draft
- They pass results to each other automatically



AI Coach John FOLLOW PROITBRIDGE Strive For Better Future Repost

FRAMEWORKS

Top Agent Frameworks to Know

You do not build agents from scratch. Pick a framework based on your use case:

LangChain / LangGraph
Best for: flexible pipelines, RAG + agents combined

CrewAI
Best for: multi-agent teams, role-based collaboration

Microsoft AutoGen
Best for: coding agents, automated conversations

OpenAI Agents SDK
Best for: quick start if already using OpenAI

Beginner tip: start with LangChain - most tutorials, biggest community.


AI Coach John FOLLOW
PROITBRIDGE Strive For Better Future
Repost

DECISION GUIDE

When Should You Use an Agent?

Agents are powerful but **overkill for simple tasks**. Use this guide to decide:

USE AN AGENT WHEN...

- ✓ The task has multiple unknown steps
- ✓ You need real-time data (search, APIs)
- ✓ The answer depends on earlier results
- ✓ You want it to run automatically on its own

SKIP THE AGENT WHEN...

- × A single prompt gives a good enough answer
- × Steps are fixed and predictable (use a chain)
- × Speed matters more than intelligence

Rule: if you can write the steps yourself, use a chain. If you *cannot predict* the steps, use an agent.



AI Coach John FOLLOW

PROITBRIDGE Strive For Better Future

Repost

PITFALLS

5 Beginner Mistakes to Avoid

Most people building their first agent hit the same walls. Dodge these early:

1. **Infinite loops** - always set `max_iterations`
2. **Vague tool descriptions** - agent picks the wrong tool
3. **No error handling** - one bad tool call crashes everything
4. **Too many tools** - confuses the LLM. Keep it under 10.
5. **Skipping verbose mode** - you cannot debug what you cannot see

```
runner = AgentExecutor(
    agent=agent, tools=tools,
    max_iterations=10,          # prevents infinite loop
    handle_parsing_errors=True, # catch bad outputs
    verbose=True                # watch every step
)
```



AI Coach John FOLLOW | PROITBRIDGE Strive For Better Future | Repost

GLOSSARY

Agentic AI Vocab Cheat Sheet

Bookmark this. Every term you will hear in the wild, explained in one sentence.

`Agent`	An LLM that loops, decides, and acts using tools
`Tool`	A function the agent can call (search, calculator...)
`ReAct`	Think, Act, Observe loop pattern
`Executor`	The runtime that runs the agent loop
`Prompt template`	Instructions that tell the agent HOW to behave
`Memory`	Storage so the agent remembers past steps
`Multi-agent`	Multiple agents collaborating on one goal
`Orchestrator`	The manager agent that delegates to sub-agents
`Grounding`	Connecting the agent to real, current data


AI Coach John FOLLOW
PROITBRIDGE Strive For Better Future
Repost

SUMMARY

Your Agentic AI Roadmap

Everything in this carousel, in order. Follow this path and you will have a working agent today.

```
1.  Understand what an agent is          <- concept
2.  Learn the 4 building blocks          <- brain/tool/memory/goal
3.  Study the Think, Act, Observe loop   <- ReAct
4.  Build your first agent (15 lines!)   <- hands-on
5.  Create custom tools                  <- @tool decorator
6.  Add memory                           <- remember context
7.  Add planning for complex goals       <- Plan-and-Execute
8.  Connect multiple agents              <- CrewAI / LangGraph
9.  Pick the right framework             <- LangChain to start
10. Know when NOT to use an agent        <- avoid overkill
11. Avoid the 5 common pitfalls          <- max_iterations etc.
12. Learn the vocabulary                 <- talk the talk
```

Save it. Share it. Start Building Today.

Follow **AI Coach John** for more AI content



AI Coach John FOLLOW

PROITBRIDGE Strive For Better Future

Repost